

UNIT II

CONTROL STRUCTURES AND STRINGS

Course Objective – To understand the control structures and string concepts.

Course Outcome – To develop program using control structure and simple string processing.

Bloom's Taxonomy Level – K1, K2, K3

PART A

S.No.	Question	BTL
1.	What are control structures? What are the different control structures used in Python? Control Structures are the blocks that analyze variables and choose directions in which to go based on given parameters. They can alter the control flow of a program execution. The primary types of control structures used in Python: - Sequential structure – default mode of execution - Selection structure - used for decisions and branching - Repetition structure - used for looping, i.e., repeating a piece of code multiple times	K1
2.	What is sequential structure? This control structure represents the linear and sequential execution of code, where statements are executed one after another in the order they appear. This is the simplest type of programming control structure and forms the foundation of most programs. Eg. <pre>a = 20 b = 10 c = a * b print(f"The product of {a} and {b} = {c}")</pre> OUTPUT The product of 20 and 10 = 200	K1
3.	What is selection structure? List the different selection control statements in Python. This control structure enables a program to choose between two or more paths of execution based on specific conditions. Several conditions are tested and instructions are executed based on which condition is true. The statements used for selection structure are also called as decision control statements or branching statements. They are - if statement - if-else statement - if – elif - else ladder statement	K1

	nested if statement	
4.	Write a python program that accepts two numbers, finds and displays the bigger one of them. <pre>A = int(input("Enter the first number")) B = int(input("Enter the second number")) if(A>B): print(A, "is bigger") else: print(B, "is bigger")</pre> <u>Output</u> Enter the first number80 Enter the second number90 90 is bigger	K3
5.	Write a Python program that allows a withdrawal transaction. Assume an available balance amount of Rs. 10,000. <pre>bal = 10000 amt = int(input("Enter the amount to withdraw: ")) if (amt >= bal): print("Insufficient balance. Withdrawal not possible.") else: bal -= amt print("Your balance amount = Rs. ", bal)</pre> <u>Output:</u> Enter the amount to withdraw: 8000 Your balance amount = Rs. 2000	K3
6.	What is iterative structure? List the different iteration statements in Python. This control structure allows certain blocks of code to be executed repeatedly as long as a condition remains true. Iteration statements in python include the following: - for loop - while loop	K2
7.	Write the syntax and usage of for loop. A for loop is a control flow statement that executes code repeatedly for a particular number of iterations. In this control flow statement, the keyword used is for. The 'for' loop is used when the number of iterations is already known. It has three forms of usage. (i) for var in range([start],end,[step]): <pre>statement block</pre> (ii) for var in list: <pre>statement block</pre>	K2

	(iii) for var in list: statement block 1 else: statement block 2 Usage: • When it is used with range() function the iterator variable var takes values from start_value to stop_value – 1. • When it is used with list / tuple, the iterator variable var takes values, one by one from the list / tuple. • Used to repeat a set of statements for a finite number of times. • Used to iterate through a string, list and tuple.	
8.	Write the syntax of while loop and its usage. It is a loop that executes a single statement or a group of statements for the given true condition. The keyword used to represent this loop is "while". A "while" loop is used when the number of iterations is unknown. It has two forms of usage. (i) while condition: statement block (ii) while condition: statement block 1 else: statement block 2 Usage: • Used to repeat a set of statements till a condition remains True. • Used to repeat a set of statements for a finite number of times. • Used to iterate through a string, list and tuple.	K1
9.	What is pass statement? ‘pass’ is a null statement. It is like a comment, but it is not ignored by the interpreter. When the user does not know what code to write, user simply places a ‘pass’ at that line. It can be placed where empty code is not allowed, like in loops, function definitions, class definitions, or in if statements. It can be used as a placeholder for future code. Syntax: pass	K2
10.	Can loop statements have else clause? If yes, when will it be executed? Yes. Python allows the else keyword to be used with the ‘for’ and ‘while’ loops too. The else block appears after the body of the loop. The statements in the else block will be executed after all iterations are completed. Eg. n = 4 for i in range(1, n): print(i)	K2

	else: print("PYTHON")											
11.	Differentiate between break and continue statements. <table><tr><th>Break</th><th>Continue</th></tr><tr><td>The break statement is used to terminate the execution of the nearest enclosing loop in which it appears.</td><td>The continue statement is used to stop the current iteration of the loop and continues with the next iteration</td></tr><tr><td>Syntax: break</td><td>Syntax: continue</td></tr><tr><td>It causes a forward jump in program execution.</td><td>It causes a backward jump in program execution.</td></tr><tr><td>Example i = 1 while(i <= 10): print(i, end="") if (i==5): break i=i+1 print("Done") Output: 1 2 3 4 5 Done</td><td>Example for i in range(1, 10): if (i % 5 == 0): continue print(i, end="") print("Done") Output: 1 2 3 4 6 7 8 9 Done</td></tr></table>	Break	Continue	The break statement is used to terminate the execution of the nearest enclosing loop in which it appears.	The continue statement is used to stop the current iteration of the loop and continues with the next iteration	Syntax: break	Syntax: continue	It causes a forward jump in program execution.	It causes a backward jump in program execution.	Example i = 1 while(i <= 10): print(i, end="") if (i==5): break i=i+1 print("Done") Output: 1 2 3 4 5 Done	Example for i in range(1, 10): if (i % 5 == 0): continue print(i, end="") print("Done") Output: 1 2 3 4 6 7 8 9 Done	K3
Break	Continue											
The break statement is used to terminate the execution of the nearest enclosing loop in which it appears.	The continue statement is used to stop the current iteration of the loop and continues with the next iteration											
Syntax: break	Syntax: continue											
It causes a forward jump in program execution.	It causes a backward jump in program execution.											
Example i = 1 while(i <= 10): print(i, end="") if (i==5): break i=i+1 print("Done") Output: 1 2 3 4 5 Done	Example for i in range(1, 10): if (i % 5 == 0): continue print(i, end="") print("Done") Output: 1 2 3 4 6 7 8 9 Done											
12.	State the differences between continue and pass statement with an example. <table><tr><th>Continue</th><th>Pass</th></tr><tr><td>The continue statement is used to ignore the remaining statements in the current iteration of the loop and moves the control back to the start of the loop.</td><td>The pass statement is used when a statement is required syntactically.</td></tr><tr><td>It stops the current iteration.</td><td>It does nothing.</td></tr><tr><td>Syntax: continue</td><td>Syntax: pass</td></tr><tr><td>Example for i in range(1, 10): if (i % 5 == 0): continue print(i, end="") print("Done") Output: 1 2 3 4 6 7 8 9 Done</td><td>Example for i in range(1, 10): if (i % 5 == 0): pass print(i, end="") print("Done") Output: 1 2 3 4 6 7 8 9 Done</td></tr></table>	Continue	Pass	The continue statement is used to ignore the remaining statements in the current iteration of the loop and moves the control back to the start of the loop.	The pass statement is used when a statement is required syntactically.	It stops the current iteration.	It does nothing.	Syntax: continue	Syntax: pass	Example for i in range(1, 10): if (i % 5 == 0): continue print(i, end="") print("Done") Output: 1 2 3 4 6 7 8 9 Done	Example for i in range(1, 10): if (i % 5 == 0): pass print(i, end="") print("Done") Output: 1 2 3 4 6 7 8 9 Done	K3
Continue	Pass											
The continue statement is used to ignore the remaining statements in the current iteration of the loop and moves the control back to the start of the loop.	The pass statement is used when a statement is required syntactically.											
It stops the current iteration.	It does nothing.											
Syntax: continue	Syntax: pass											
Example for i in range(1, 10): if (i % 5 == 0): continue print(i, end="") print("Done") Output: 1 2 3 4 6 7 8 9 Done	Example for i in range(1, 10): if (i % 5 == 0): pass print(i, end="") print("Done") Output: 1 2 3 4 6 7 8 9 Done											

13.	Point out the errors in the following codes: (i) for i in range(1, 10, 2.0): print(i) (ii) while j <= 10: print(j) j++ (i) TypeError: 'float' object cannot be interpreted as an integer – float values cannot be used in range function. (ii) 2 errors: (a) SyntaxError: invalid syntax - ++ is not an operator in Python	K2
14.	Explain string comparison with an example. The comparison operator works on string to check if two strings are equal. <pre>>>>PIN='1234' >>>if PIN=='1234': print("Correct PIN") Correct PIN</pre>	K3
15.	What is meant by string slicing? Give examples. A substring of a string is obtained by taking a slice. The operator [n:m] returns the part of the string from the nth character to the mth character, including the first but excluding the last. Example: <pre>>>>book='Problem Solving and Python Programming' >>>print(book[0:7]) Problem >>>print(book[21:27]) Python</pre>	K2
16.	Mention the use of the split function in Python. The use of split function in Python is that it breaks a string into shorter strings using the defined separator. It gives a list of all words present in the string Syntax: string.split(separator, maxsplit) separator – Optional parameter - Specifies the separator to use when splitting the string. By default any whitespace is a separator maxsplit – Optional parameter - Specifies how many splits to do. Default value is -1, which is "all occurrences" Eg. <pre>txt = "hello world" x = txt.split(" ") print(x)</pre> Output: ['hello', 'world']	K2
17.	What is a Negative Indexing in Python? Negative Indexing is used to in Python to begin slicing from the end of the string i.e. the last. Slicing in Python gets a sub-string from a string. The slicing range is set as parameters i.e. start, stop, and step.	K2

	Eg. <pre>myStr = 'Python Programming!' print("String = ", myStr) print("String after slicing (negative indexing) = ", myStr[-4 : -1])</pre> Output: String after slicing (negative indexing) = ing																											
18.	What are the two operators that are used in string functions? The 2 operators used in string functions are ‘in’ and ‘not in’. They can be used to test whether a substring is a member of a string or not. <pre>>>>'t' in 'Python' True >>>'p' in 'Python' False >>>'thon' not in 'Python' False >>>'then' not in 'Python' True</pre>	K2																										
19.	What is the use of str.upper() and str.lower() functions in string?? The functions str.upper() and str.lower() will return a string with all the letters of original string converted to upper or lower case letters. <pre>>>>ss = 'Python Programming' >>>print(ss.upper()) PYTHON PROGRAMMING >>>print(ss.lower()) python programming</pre>	K2																										
20.	List few string methods and their purpose, used in Python. <table><thead><tr><th>Method</th><th>Description</th></tr></thead><tbody><tr><td>capitalize()</td><td>Converts the first character to upper case</td></tr><tr><td>find()</td><td>Searches the string for a specified value and returns the position of where it was found</td></tr><tr><td>index()</td><td>Searches the string for a specified value and returns the position of where it was found.</td></tr><tr><td>isalpha()</td><td>Returns True if all characters in the string are in the alphabet</td></tr><tr><td>isdigit()</td><td>Returns True if all characters in the string are digits</td></tr><tr><td>isnumeric()</td><td>Returns True if all characters in the string are numeric</td></tr><tr><td>islower()</td><td>Returns True if all characters in the string are lower case</td></tr><tr><td>isupper()</td><td>Returns True if all characters in the string are upper case</td></tr><tr><td>lower()</td><td>Converts a string into lower case</td></tr><tr><td>upper()</td><td>Converts a string into upper case</td></tr><tr><td>replace()</td><td>Returns a string where a specified value is replaced with a specified value</td></tr><tr><td>strip()</td><td>Returns a trimmed version of the string</td></tr></tbody></table>	Method	Description	capitalize()	Converts the first character to upper case	find()	Searches the string for a specified value and returns the position of where it was found	index()	Searches the string for a specified value and returns the position of where it was found.	isalpha()	Returns True if all characters in the string are in the alphabet	isdigit()	Returns True if all characters in the string are digits	isnumeric()	Returns True if all characters in the string are numeric	islower()	Returns True if all characters in the string are lower case	isupper()	Returns True if all characters in the string are upper case	lower()	Converts a string into lower case	upper()	Converts a string into upper case	replace()	Returns a string where a specified value is replaced with a specified value	strip()	Returns a trimmed version of the string	K3
Method	Description																											
capitalize()	Converts the first character to upper case																											
find()	Searches the string for a specified value and returns the position of where it was found																											
index()	Searches the string for a specified value and returns the position of where it was found.																											
isalpha()	Returns True if all characters in the string are in the alphabet																											
isdigit()	Returns True if all characters in the string are digits																											
isnumeric()	Returns True if all characters in the string are numeric																											
islower()	Returns True if all characters in the string are lower case																											
isupper()	Returns True if all characters in the string are upper case																											
lower()	Converts a string into lower case																											
upper()	Converts a string into upper case																											
replace()	Returns a string where a specified value is replaced with a specified value																											
strip()	Returns a trimmed version of the string																											

PART B

S. No	Question	BTL
1.	Explain in detail about the different conditional control statements / Selection Control statements / decision making with an example. (16 Marks)	K2
2.	What are the different looping construct available in python? Explain each of them in detail with relevant example. (16 Marks)	K2
3.	Write in detail about the different unconditional statements in Python. Give example program for each. (16 Marks)	K2
4.	a) Design a simple calculator using if ...elif construct. (16 Marks) b) Write a Python program to check the prime number using while loop.	K3
5.	a) Write a python program to print factorial of a given number (16 Marks) b) Implement a Python program to find the sum of n even natural numbers.	K3
6.	a) Develop a program to find the largest among three numbers. (16 Marks) b) Write a python program to find the given year is leap or not.	K3
7.	Discuss about the following operations related to strings in Python. (8 Marks) (a) String comparison (8 Marks) (b) String slicing (c) String splitting (d) String stripping	K2
8.	Explain the different string manipulation functions available in Python with example. (16 Marks)	K2